ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) | 979-8-3503-6874-1/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICASSP49660.2025.10887721

Adaptive Lossless Compression for Genomics Data by Multiple (s, k)-mer Encoding and XLSTM

Hui Sun[1, 2, †], Yanfeng Ding[1, †], Liping Yi[1, 2], Huidong Ma[1], Haonan Xie[3], Gang Wang[1, *], Xiaoguang Liu[1, *]
[1]College of Computer Science, NBJL, ICIC, TMCC, SysNet, DISSec, GTIISC, Nankai University, Tianjin 300350, China
[2]College of Computing and Data Science (CCDS), Nanyang Technological University, Nanyang 639798, Singapore
[3]School of Electrical Engineering, Guangxi University, Nanning 530004, China

***Abstract*—Learning-based lossless compressors have been validated to have competitive advantages in genomics data (GD) compression. However, learning-based GD-dedicated compressors typically need to be pre-trained on multi-source data and then are directly used to compress another target data, we denote them as static compressors, and they often face two challenges: limited compression ratios and bad-performed generalization due to data distribution variations. To solve these problems, we propose AGDLC, a novel Adaptive Genomics Data Lossless Compressor. It includes two critical designs: 1) We design a multiple (s, k)-mer mixer for extracting GD redundancy from multiple dimensions to improve compression ratios. 2) We introduce a recently popular XLSTM model as the backbone, which adaptively compresses GD while updating parameters, without pre-training, improving compression ratios and compression generalization at the same time. We compare AGDLC with 13 baselines on 7 real-world datasets, and the experimental results demonstrate that it achieves the best compression ratio with an average improvement of 2.162%-69.436%. The codes can be found at https://github.com/dingyanfeng/AGDLC.**

***Index Terms*—lossless compression, genomics data, learning-based compressor, (s, k)-mer encoding, XLSTM.**

I. Introduction

Genomics Data (GD), consisting of $\{A, C, G, T\}$, plays a crucial role in forensic identification and precision healthcare services [1]–[4]. However, the massive volume of GD limits their transmission, sharing, and storage [5], [6]. Compressing them can address these issues.

There are two main types of GD compression methods: traditional and learning-based. Traditional compressors [7]–[10] have suboptimal compression effects, since they can not fully utilize GD's redundant characteristics. Learning-based compressors [11]–[14] have become mainstream in industry and academia, as they effectively exploit GD's redundant characteristics (frequent repetitions of symbols) by sophisticated context-based modeling. However, these compressors typically first pre-train a neural-network-based (NN-based) model on a multi-source GD and then directly apply it to compress another target GD with a similar data distribution as the multi-source GD. During the process of the target GD compression, the NN-based model is not be trained, so we denote it as static learning-based compressors, which introduces two challenges:

- **Limited Compression Ratio**. The static learning-based GD compressors suffer from inferior compression ratios, especially when the model is pre-trained insufficiently.
- **Bad-performed Generalization.** The static learning-based GD compressors have bad generalization, because they perform well only when the target GD to be compressed has a similar distribution to the pre-trained GD.

To solve them, we propose a novel **A**daptive **G**enomics **D**ata **L**ossless **C**ompressor (**AGLDC**) with three contributions:

- We design a multiple (s, k)-mer mixer to extracts GD's redundancy features from multiple dimensions for improving compression ratios.
- We introduce a XLSTM model with a strong contextual modeling capability as the NN's backbone, and adaptively train it during GD compression, enabling the improvement of compression ratios and generalization.
- We evaluate AGLDC and 13 baselines on 7 real-world datasets to demonstrate that AGLDC achieves the best compression ratio with average 2.162%-69.436% improvements compared with baselines.

II. Related Work

GD compressors consists of traditional and learning-based. **Traditional Compressors:** These lossless methods are primarily based on the Burrows Wheeler Transform (BWT), dictionaries, bit-encoding, or static contex-based modeling [15]–[17], such as NAF [7], MFCompress [8], GenoZip [9], SPRING [10], and etc. These methods do not fully utilize GD redundancy, leading to sub-optimal compression ratios.

Learning-based Compressors: They often use a static pre-trained model to compress GD losslessly. DeepDNA [18] uses convolutional neural network (CNN), long short-term memory (LSTM), and fully connected layer. DNA-BiLSTM [19] improves DeepDNA by using Bidirectional LSTM (BiLSTM) and attention mechanism. CompressBERT [13] employs BERT [20] to achieve global position coding for complete GD sequences. GeneFormer [12] uses transformer with multi-level-grouping. GeCo3 [21] and JAVRIS3 [22] design NN-based multiple-context-model mixtures. Although considering GD's redundancy information, they need to guarantee that the compressed GD has a similar distribution to the pre-trained GD, resulting in inferior compression ratio and generalization.

To achieve satisfactory compression ratio and generalization, we propose AGLDC with multiple (s, k)-mer encoding

† Equal contributions.
* Corresponding authors: {wgzwp, liuxguang}@nbjl.nankai.edu.cn.
This research is supported in part by the National Science Foundation of China under Grant 62272252 & 62272253, and the Fundamental Research Funds for the Central Universities.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:25:54 UTC from IEEE Xplore. Restrictions apply.

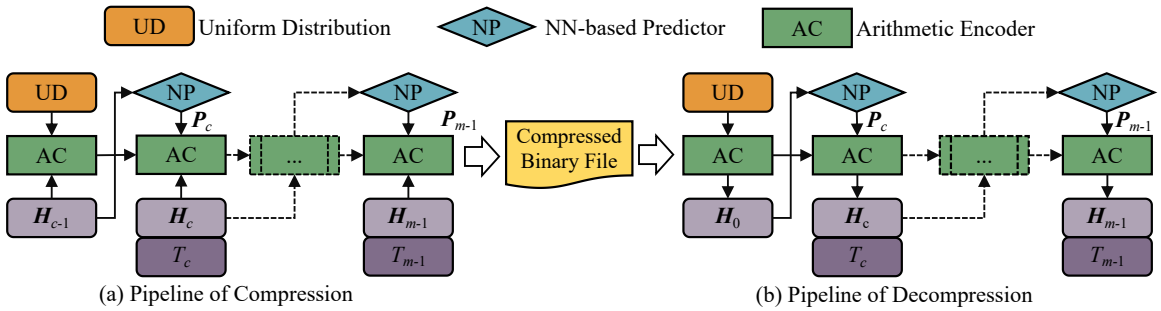


Fig. 1. Pipelines of compression and decompression in AGDLC.

and XLSTM. Multiple (s, k)-mer encoding extracts redundancy features from multiple dimensions, and the XLSTM model is trained during GD compression, without pre-training, both of them effectively improve compression ratio and generalization.

III. METHODOLOGY

A. Overview

Fig. 1 shows the compression and decompression pipelines of AGLDC.

Compression. As shown in Fig. 1(a), AGLDC first feeds c historical GD symbols $\boldsymbol{H}_i = \{S_{i-c}, ..., S_i\}$ and a target symbol $T_i = S_{i+1}$ into an NN-based predictor to generate a probability vector $\boldsymbol{P}_i = P(T_i|\boldsymbol{H}_i)$, $i = \{c, ..., m-2\}$, m is the length of the GD sequence. Then, $\boldsymbol{P}_i$ is input into an arithmetic encoder used in [11] to generate compressed binary file. In this step, an accurate $\boldsymbol{P}_i$ can be compressed to a smaller compressed binary files [23]–[25], so we design a XLSTM-based predictor to enhance prediction capability, as shown in Fig. 3. For the initial c symbols $\boldsymbol{H}_{c-1} = \{S_0, ..., S_{c-1}\}$, we adopt a uniform distribution [11] for compressing. Algorithm 1 illustrates the adaptive compression process.

Decompression. As shown in Fig. 1(b), during decompression, AGLDC again uses the NN-based model to recover original GD symbols. We ensure the consistency of the initialized NN parameters in the compression and decompression processes to achieve lossless decompression.

Under the above two processes, the prediction capability of the NN-based predictor determines the compression ratio, which is carefully designed with two components: a (s, k)-mer mixer and a XLSTM-based prediction module.

B. (s, k)-mer Mixer

We design a mixer consisting of α numbers of (s, k)-mer encoders to extract GD's redundancy from multiple dimensions. As shown in Fig. 2, it involves the following steps (subfigs):

(a). It first map each historical symbol ($\in \{A, C, G, T\}$) in $\boldsymbol{H}_i$ to a number representations $\in \{1, 2, 3, 4\}$.

(b). For α (s, k)-mer encoders, the stride s is fixed for overlap calculation and the sliding window length is varied as $\boldsymbol{K} \in \{k_0, ..., k_{\alpha-1}\}$ for rich features extraction.

Algorithm 1: AGDLC Adaptive Compression Process

Input: sequence S; context-length c; (s, k)-mer encoding parameters s, $\boldsymbol{K}$, and α; XLSTM module parameters β, γ, and h
Output: compressed binary file $*.agdlc$

initializing $\mathcal{M}_{adaptive}$ model using β, γ, and h ;
initializing arithmetic encoder AC ;
for $i = 0$ **to** $c - 1$ **do**
  generating uniform distribution for the i-th symbol S_i;
  feeding uniform distribution into AC for compressing;
for $i = c$ **to** $m - 1$ **do**
  getting historical symbols $\boldsymbol{H}_i$ and target symbol T_i;
  multiple (s, k)-mer encoding $\boldsymbol{H}_i$ and T_i to get $\boldsymbol{E}_i$ via s, $\boldsymbol{K}$, α;
  calculating $\boldsymbol{P}_i$ using XLSTM-based $\mathcal{M}_{adaptive}$ and $\boldsymbol{E}_i$;
  feeding $\boldsymbol{P}_i$ into AC for compressing;
  back-propagating update $\mathcal{M}_{adaptive}$ using $\boldsymbol{T}_i$ and $\boldsymbol{P}_i$;
generating compressed binary file $*agdlc$ as output;

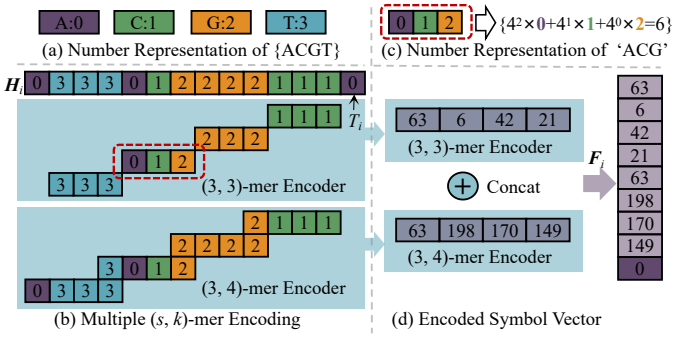


Fig. 2. The workflow of the (s, k)-mer mixer. $\boldsymbol{H}_i$, and T_i, $\boldsymbol{F}_i$ represent historical symbols, target symbol, and encoded features, respectively.

(c). Each (s, k)-mer encoder encodes the elements of each sliding window to be a number, such as "012" is encoded to "6" by calculating $4^2 \times 0 + 4^1 \times 1 + 4^0 \times 2$.

(d). The encoded vectors from α (s, k)-mer encoders are spliced to be a long encoding $\boldsymbol{F}_i$ with length v, and then it is processed by an embedding layer to generate the final feature embedding vector $\boldsymbol{E}_i$ with dimension γ, which is fed into the prediction module for predicting the probability of the target symbol T_i.

Among these four steps, the context-length c, the length v of encoded vector, the number α of (s, k)-mer encoders, the stride s, and the sliding window $\boldsymbol{K} = \{k_0, ..., k_{\alpha-1}\}$ satisfy:

$$c = (\lceil \frac{v}{\alpha} \rceil - 1) \times s + max\{k_0, ..., k_{\alpha-1}\}. \quad (1)$$

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:25:54 UTC from IEEE Xplore. Restrictions apply.

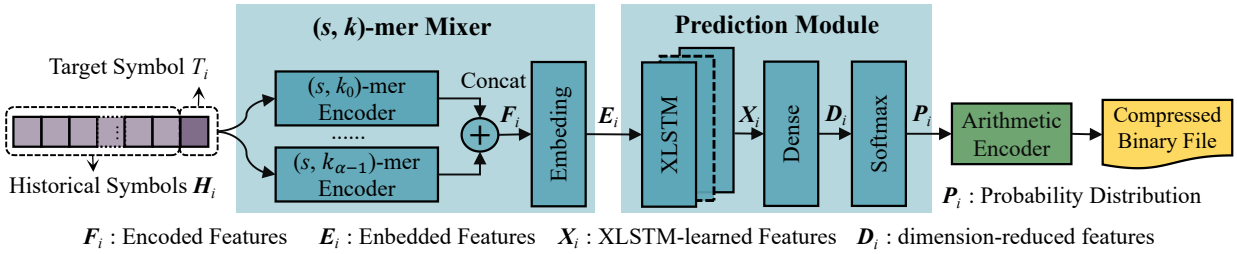


Fig. 3. The workflow of the XLSTM-based prediction module.

C. Prediction Module

As shown in Fig. 3, the generated feature embedding vector $\boldsymbol{E}_i$ from (s,k)-mer mixer is input into the designed prediction module to predict the probability vector of the target symbol T_i. It is designed to involve three parts: 1) a XLSTM block for extracting features, 2) a dense block for feature scaling, and 3) a softmax layer for probability output.

XLSTM Block. It takes the feature embedding vector $\boldsymbol{E}_i$ as input and extracts feature $\boldsymbol{X}_i$ with dimension $v \times k^{|\Theta|}$ as output, where $i = \{c, ..., m-1\}$ and $|\Theta|$ denote the GD alphabet size. The XLSTM model with β layers is a state-of-the-art time series model with two strengths: 1) It improves the forgetting processes of RNN-based models that can better capture context dependencies. 2) The gate mechanism in XLSTM can extract long-term dependencies more effectively than Transformer with self-attention mechanisms.

Dense Block. It is a h-dimension hidden layer. It is fed into the extracted feature $\boldsymbol{X}_i$ from the XLSTM block to obtain the dimension-reduced feature $\boldsymbol{D}_i$ with dimension $|\Theta|$.

Softmax Block. This block is a softmax function. It maps the dimension-reduced feature $\boldsymbol{D}_i$ into the probability distribution $\boldsymbol{P}_i = P(T_i|\boldsymbol{H}_i)$ with dimension $|\Theta|$. Each $P(T_{i,j}|H_i)$ $(j = \{0, 1, ..., |\Theta|-1\})$ is calculated as:

$$P(T_{i,j}|\boldsymbol{H}_i) = log(\frac{e^{D_{i,j}}}{\sum_{p=0}^{|\Theta|-1} e^{D_{i,p}}}). \tag{2}$$

where e is Euler's number. The probability vector $\boldsymbol{P}_i$ is input into an arithmetic encoder to obtain a compressed binary file.

Model Training. Different from static pre-trained learning-based compressors [12]–[14], AGDLC adaptively trains the prediction model while performing GD compression. During model training, we use the Cross-Entropy (CE) loss function [11], [26], which is defined as:

$$\mathcal{L}(T', P) = \sum_{i=c}^{m-1} CE(\boldsymbol{T}'_i, \boldsymbol{P}_i) = \sum_{i=c}^{m-1} \sum_{j=0}^{|\Theta|-1} (\boldsymbol{T}'_{ij} log \frac{1}{\boldsymbol{P}_{ij}}). \tag{3}$$

where $\boldsymbol{T}'_i$ and $\boldsymbol{P}_i$ are the probability vectors of the encoded ground truth $\boldsymbol{T}_i$ and the model prediction, respectively.

IV. EXPERIMENTS

In this section, we compare AGDLC with 13 baselines on 7 real-world datasets under a linux server with 2*Intel Xeon Silver 4310 CPUs, 4*NVIDIA GeForce RTX 4090 GPUs (24 GB), and 128 GB DDR5 memory.

A. Settings

Datasets. We use 7 real-world datasets from [5], [27]–[30] with a size of 49,635,328 bytes, including various species: AeCa (Aeropyrum camini), PlFa (Plasmodium falciparum), WaMe (Wolbachia endosymbiont), HePy (Helicobacter pylori), HaHi (Haloarcula hispanica), ArTh (Arabidopsis thaliana), and EsCo (Escherichia coli).

Baselines. We compare AGDLC with 13 methods, including 6 traditional general compressors: LZMA2 [31], XZ [32], PPMD [33], PBzip2 [34], Gzip [35], and SnZip [36]; 3 traditional GD-dedicated compressors: NAF [7], GenoZip [9], and SPRING [10]; 3 NN-based general compressors: DZIP [11], DeepZip [37], and LSTM-Compress [38]; and 1 NN-based GD-dedicated compressor: JAVRIS3 [22].

Metric. The compression ratio (CR) is a common metric in long-term backup scenarios [39]–[43], which is defined as:

$$CR = \frac{Compressed_File_Size_in_bytes}{Original_File_Size_in_bytes} \times 8 \ \ (bits/base). \tag{4}$$

Hyperparameters. We configure the hyperparameters of baselines following their papers. For the hyperparameters in AGDLC, we set $c = 65$, $\alpha = 2$, $s = 2$, $\boldsymbol{K} \in \{2, 3\}$, $\gamma = 16$, $\beta = 1$, and $h = 64$. During model training, we use an Adam optimizer [44] with parameters $\beta_1 = 0$ and $\beta_2 = 0.999$.

B. Comparison Experiments

Table I shows that AGLDC with $s = 2$ and $\boldsymbol{K} \in \{2, 3\}$ achieves the best average compression ratio. Compared with 13 baselines, it achieves the average CR improvement $(\frac{CR(Baseline)-CR(Ours)}{CR(Baseline)} \times 100\%)$ ranging from 2.162% to 69.436%. This advantage is attributed to two points: 1) the utilization of multiple (s,k)-mer encoders helps AGLDC to incorporate GD redundancy features from multiple dimensions, 2) the introduced state-of-the-art XLSTM-based prediction model enhances probability prediction capability.

C. Ablation Study

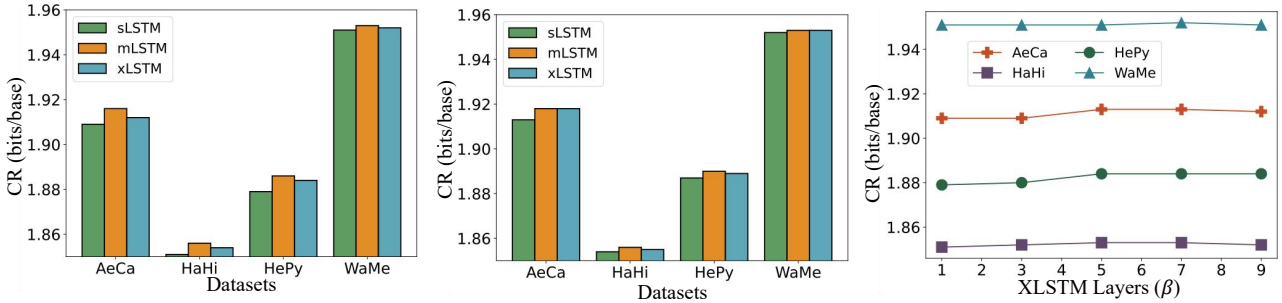
We conduct ablation study for AGLDC with different $(s = 1)$ combinations. The results in Table I demonstrate that AGLDC with multiple (s,k)-mer encoders performs better than with one (s,k)-mer encoder. This demonstrates the necessity of the (s,k)-mer mixer.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:25:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I

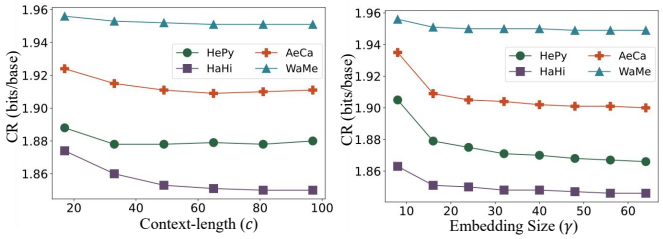
COMPRESSION RATIOS OF ALL METHODS. LOWER CR MEANS BETTER COMPRESSION EFFECT. "BOLDFACE" DENOTES TOP-3 RESULTS.

Methods/Datasets	AeCa	PlFa	WaMe	HePy	HaHi	ArTh	EsCo	Average	Improvement
Genozip [9]	1.976	1.864	1.974	1.932	1.928	**1.894**	1.986	1.936	2.162%
Spring [10]	2.008	1.859	1.988	1.915	1.937	1.934	1.976	1.945	2.614%
NAF [7]	1.992	1.874	1.965	1.988	1.954	1.929	1.999	1.957	3.211%
JAVRIS3 [22]	2.026	1.935	2.036	1.980	2.015	**1.902**	1.997	1.984	4.535%
LZMA2 [31]	2.076	1.867	2.070	1.990	2.025	1.905	2.043	1.997	5.116%
XZ [32]	2.082	1.869	2.075	1.993	2.029	1.907	2.045	2.000	5.279%
PPMD [33]	2.078	1.906	2.109	1.990	2.035	2.008	2.063	2.027	6.540%
DeepZip [37]	2.504	1.901	2.033	2.409	2.067	1.935	2.099	2.136	11.295%
PBzip2 [34]	2.153	2.094	2.169	2.123	2.120	2.138	2.158	2.136	11.327%
DZIP [11]	2.519	1.902	2.037	2.429	2.073	1.936	2.106	2.143	11.605%
Gzip [35]	2.236	2.121	2.250	2.177	2.203	2.186	2.240	2.202	13.962%
SnZip [36]	3.724	3.641	3.742	3.686	3.710	3.696	3.728	3.704	48.853%
Lstm-compress [38]	4.496	6.999	7.395	3.909	6.457	7.375	6.757	6.198	69.436%
AGDLC ($s=1, K=1$)	1.953	1.844	1.959	1.920	1.881	1.907	1.961	1.918	1.222%
AGDLC ($s=2, K=2$)	**1.923**	**1.836**	**1.952**	**1.899**	**1.861**	**1.902**	**1.945**	**1.903**	0.428%
AGDLC ($s=2, K=3$)	**1.925**	**1.835**	**1.953**	**1.894**	**1.856**	**1.902**	**1.941**	**1.901**	0.338%
AGDLC ($s=2, \boldsymbol{K} \in \{2,3\}$)	**1.909**	**1.832**	**1.951**	**1.879**	**1.851**	**1.901**	**1.938**	**1.894**	-



(a) CR with 1-layer XLSTM (b) CR with 2-layer XLSTM (c) The Impact of XLSTM Layers

Fig. 4. Impacts of types and numbers of the XLSTM layers on compression ratios.



(a) The Impact of Context-length (b) The Impact of Embedding Size

Fig. 5. Impacts of context-length and embedding size on compression ratios.

D. The Impact of XLSTM Layer β

The XLSTM layers include two special types: sLSTM with a memory structure incorporating new memory fusion techniques, and mLSTM introducing matrix memory to enable parallel processing [45]. We first compare how the the layer types of XLSTM affect CR. Fig. 4(a∼b) show the results of sLSTM, mLSTM, and XLSTM (sLSTM + mLSTM) with layer number $\beta = 1$ and $\beta = 2$. We can see that the sLSTM achieves the best CR on 4 datasets for both $\beta = 1$ and $\beta = 2$, which is also the default XLSTM structure in our experiments.

Fig. 4(c) shows how the number of sLSTM layers affects CR. We find that CR and β maintain a positive correlation, so choose $\beta = 1$ with the best CR by default.

E. The Impact of Context-length c

Fig. 5(a) shows that a larger context-length c has a better CR, because a larger c captures more local redundancies. This trend tends to be stable at $c = 65$, which is set to be default.

F. The Impact of Embedding Dimension γ

Fig. 5(b) shows that a larger embedding dimension γ achieves a better CR, because a more complex embedding enables the XLSTM-based model to learn richer features. Considering model complexity, we choose $\gamma = 16$ by default.

V. CONCLUSION AND FUTURE WORK

This paper proposed AGLDC, a novel lossless adaptive GD-dedicated compressor with two key designs: 1) A multiple (s, k)-mer mixer is designed to extract redundancy features from multiple dimensions to improve compression ratios. 2) A XLSTM-based prediction model is designed to be trained adaptively during GD compression, enabling the improvements of both compression ratios and addressing. Experiments on 7 datasets with 13 baselines validate that AGDLC achieves the best compression ratio. In future work, it is promising to further the structure XLSTM-based prediction model to improve the throughput of compression and decompression.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:25:54 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] Kelvin V Kredens, Juliano V Martins, Osmar B Dordal, Mauri Ferrandin, Roberto H Herai, Edson E Scalabrin, and Braulio C Avila. Vertical lossless genomic data compression tools for assembled genomes: A systematic literature review. *Plos one*, 15(5):e0232942, 2020.

[2] Hui Sun, Yingfeng Zheng, Haonan Xie, Huidong Ma, Xiaoguang Liu, and Gang Wang. Pmffrc: a large-scale genomic short reads compression optimizer via memory modeling and redundant clustering. *BMC bioinformatics*, 24(1):454, 2023.

[3] Hui Sun, Huidong Ma, Yingfeng Zheng, Haonan Xie, Meng Yan, Cheng Zhong, Xiaoguang Liu, and Gang Wang. LRCB: A comprehensive benchmark evaluation of reference-free lossless compression tools for genomics sequencing long reads data. In *2024 Data Compression Conference (DCC)*, pages 584–584. IEEE, 2024.

[4] Hui Sun, Huidong Ma, Yingfeng Zheng, Haonan Xie, Xiaofei Wang, Xiaoguang Liu, and Gang Wang. Sr2c: A structurally redundant short reads collapser for optimizing dna data compression. In *2023 IEEE 29th International Conference on Parallel and Distributed Systems (ICPADS)*, pages 60–67, 2023.

[5] Lewis Y Geer, Aron Marchler-Bauer, Renata C Geer, Lianyi Han, Jane He, Siqian He, Chunlei Liu, Wenyao Shi, and Stephen H Bryant. The ncbi biosystems database. *Nucleic acids research*, 38(suppl_1):D492–D496, 2010.

[6] Kris A Wetterstrand. Dna sequencing costs: data from the nhgri genome sequencing program (gsp). *National Human Genome Research Institute*, 2023.

[7] Kirill Kryukov, Mahoko Takahashi Ueda, So Nakagawa, and Tadashi Imanishi. Nucleotide archival format (naf) enables efficient lossless reference-free compression of dna sequences. *Bioinformatics*, 35(19):3826–3828, 2019.

[8] Armando J Pinho and Diogo Pratas. Mfcompress: a compression tool for fasta and multi-fasta data. *Bioinformatics*, 30(1):117–118, 2014.

[9] Divon Mordechai Lan, Daniel ST Hughes, and Bastien Llamas. Deep fastq and bam co-compression in genozip 15. *bioRxiv*, pages 2023–07, 2023.

[10] Shubham Chandak, Kedar Tatwawadi, Idoia Ochoa, Mikel Hernaez, and Tsachy Weissman. Spring: a next-generation compressor for fastq data. *Bioinformatics*, 35(15):2674–2676, 2019.

[11] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. Dzip: Improved general-purpose loss less compression based on novel neural network modeling. In *2021 Data Compression Conference (DCC)*, pages 153–162. IEEE, 2021.

[12] Zhanbei Cui, Tongda Xu, Jia Wang, Yu Liao, and Yan Wang. Geneformer: Learned gene compression using transformer-based context modeling. In *ICASSP 2024-2024 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 8035–8039. IEEE, 2024.

[13] Han Yang, Fei Gu, and Jieping Ye. Rethinking learning-based method for lossless genome compression. In *ICASSP 2023-2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pages 1–5. IEEE, 2023.

[14] KS Sheena and Madhu S Nair. Gencoder: A novel convolutional neural network based autoencoder for genomic sequence data compression. *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, (01):1–12, 2024.

[15] Sun Hui and Ma Huidong. Neural-network-based lossless universal compressions benchmark, 2024. [Online]. Available: https://fahaihi.github.io/NNLCB.

[16] Mikel Hernaez, Dmitri Pavlichin, Tsachy Weissman, and Idoia Ochoa. Genomic data compression. *Annual Review of Biomedical Data Science*, 2:19–37, 2019.

[17] Giacomo Chiarot and Claudio Silvestri. Time series compression survey. *ACM Computing Surveys*, 55(10):1–32, 2023.

[18] Rongjie Wang, Yang Bai, Yan-Shuo Chu, Zhenxing Wang, Yongtian Wang, Mingrui Sun, Junyi Li, Tianyi Zang, and Yadong Wang. Deepdna: A hybrid convolutional and recurrent neural network for compressing human mitochondrial genomes. In *2018 IEEE International Conference on Bioinformatics and Biomedicine (BIBM)*, pages 270–274. IEEE, 2018.

[19] Wenwen Cui, Zhaoyang Yu, Zhuangzhuang Liu, Gang Wang, and Xiaoguang Liu. Compressing genomic sequences by using deep learning. In *International Conference on Artificial Neural Networks*, pages 92–104. Springer, 2020.

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018.

[21] Milton Silva, Diogo Pratas, and Armando J Pinho. Efficient dna sequence compression with neural networks. *GigaScience*, 9(11):giaa119, 2020.

[22] Diogo Pratas. Jarvis3: an improved encoder for genomic sequences, 2024. [Online]. Available: https://github.com/cobilab/jarvis3.

[23] Glen G. Langdon Jr. An introduction to arithmetic coding. *IBM J. Res. Dev.*, 28(2):135–149, 1984.

[24] Jorma Rissanen and Glen G. Langdon Jr. Universal modeling and coding. *IEEE Trans. Inf. Theory*, 27(1):12–22, 1981.

[25] Khalid Sayood. *Introduction to data compression*. Morgan Kaufmann, 2017.

[26] Anqi Mao, Mehryar Mohri, and Yutao Zhong. Cross-entropy loss functions: Theoretical analysis and applications. In *International conference on Machine learning*, pages 23803–23828. PMLR, 2023.

[27] D Pratas, AJ Pinho, and PJSG Ferreira. Efficient compression of genomic sequences. In *2016 Data compression conference (DCC)*, pages 231–240. IEEE, 2016.

[28] Diogo Pratas, Morteza Hosseini, and Armando J Pinho. Geco2: An optimized tool for lossless compression and analysis of dna sequences. In *Practical Applications of Computational Biology and Bioinformatics, 13th International Conference*, pages 137–145. Springer, 2020.

[29] Andreas Ingo Grohmann, Johannes VS Busch, Adrian Bretschneider-Perez, Christopher Lehmann, and Frank HP Fitzek. Javris: Joint artificial visual prediction and control for remote-(robot) interaction systems. In *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, pages 835–840. IEEE, 2022.

[30] Diogo Pratas and Armando J Pinho. A DNA sequence corpus for compression benchmark. In *Practical Applications of Computational Biology and Bioinformatics, 12th International Conference*, pages 208–215. Springer, 2019.

[31] Ipavlov. A free file archiver for extremely high compression, 2024. [Online]. Available: https://www.7- zip.org.

[32] XZ. Xz official website, 2024. [Online]. Available: https://github.com/tukaani-project/xz.

[33] John G. Cleary and Ian H. Witten. Data compression using adaptive coding and partial string matching. *IEEE Trans. Commun.*, 32(4):396–402, 1984.

[34] PBzip2. pbzip2 - parallel bzip2 file compressor, 2024. [Online]. Available: https://linux.die.net/man/1/pbzip2.

[35] Gzip. Gzip official website, 2018. [Online]. Available: http://www.gzip.org/.

[36] Kubo Takehiro. Snzip, a compression/decompression tool based on snappy., 2021. [Online]. Available: https://github.com/kubo/snzip.

[37] Mohit Goyal, Kedar Tatwawadi, Shubham Chandak, and Idoia Ochoa. Deepzip: Lossless data compression using recurrent neural networks. *arXiv preprint arXiv:1811.08162*, 2018.

[38] Byron Knoll. Data compression using lstm, 2017. [Online]. Available: https://github.com/byronknoll/lstm-compress.

[39] Hui Sun, Yingfeng Zheng, Haonan Xie, Huidong Ma, Cheng Zhong, Meng Yan, Xiaoguang Liu, and Gang Wang. PQSDC: a parallel lossless compressor for quality scores data via sequences partition and Run-Length prediction mapping. *Bioinformatics*, page btae323, 05 2024.

[40] Byron Knoll. Cmix official website, 2024. [Online]. Available: https://github.com/byronknoll/cmix.

[41] Dohyeon Lee and Giltae Song. Fastqcls: a fastq compressor for long-read sequencing via read reordering using a novel scoring model. *Bioinformatics*, 38(2):351–356, 2022.

[42] Qingxi Meng, Shubham Chandak, Yifan Zhu, and Tsachy Weissman. Reference-free lossless compression of nanopore sequencing reads using an approximate assembly approach. *Scientific Reports*, 13(1):2082, 2023.

[43] Diogo Pratas and Armando J Pinho. Jarvis2: a data compressor for large genome sequences. In *2023 Data Compression Conference (DCC)*, pages 288–297. IEEE, 2023.

[44] Kingma Da. A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[45] Maximilian Beck, Korbinian Pöppel, Markus Spanring, Andreas Auer, Oleksandra Prudnikova, Michael Kopp, Günter Klambauer, Johannes Brandstetter, and Sepp Hochreiter. xlstm: Extended long short-term memory. *arXiv preprint arXiv:2405.04517*, 2024.

Authorized licensed use limited to: NANKAI UNIVERSITY. Downloaded on October 23,2026 at 11:25:54 UTC from IEEE Xplore. Restrictions apply.